

Antora Chattopadhyay

Senior product designer. I design clarity,
then own the backlog that ships it.

At Spry I own the design of the billing, eligibility and verification products that 50K+ users across 120+ clinics rely on. Before that, four years at an agency shipping 50+ websites and products for US and UK clients.

SELECTED WORK

01	Patient workspace Audit, reframe and a case-first record	03
02	Unified Billing Dashboard One product, seven configurations	10
03	SPRY Verify Eligibility and prior auth, task first	15
04	Websites and messaging 50+ sites, Agilisium to Trocco	20
05	Product ownership Backlog, PRDs, stakeholders, data	21

50K+

PLATFORM USERS

120+

CLINICS SERVED

97%

ON-TIME DELIVERY

I work where design meets the backlog.

I start with the unglamorous part of a problem: watching how people actually make decisions. Then I work with PMs and engineers to turn that into a direction and ship it. Most of my work is dense, rule-driven, data-heavy territory, where clarity and accuracy matter more than polish.

Design

Information architecture, interaction and design systems in Figma, plus working React, Tailwind and shadcn prototypes tested against real SQL and API data.

Product ownership

PRDs, my own Jira backlog, and structured reviews with engineers, PMs and clinical operations leads across US and India time zones.

Research and data

Operator interviews and usability testing, then SQL when an argument needs evidence, not opinion.

AI in the workflow

Claude with a persistent product context file, Figma Make, Lovable and Figma MCP for research synthesis, prototypes and specs. I direct and review its output critically.

- Spry Therapeutics

Senior Product Designer

US B2B SaaS for 120+ physical therapy clinics and 50K+ users, with 900K+ transactions processed YTD. Also designed batch review and payment posting, pre-authorization alerts, and a Medicare cap and KX status tracker.

SEP 2025 TO PRESENT
- Nitrous Design Studio

UX Designer, Senior UX Designer, UX Manager

50+ websites and products for US and UK clients at 97% on-time delivery. Clients include Agilisium, Apex Covantage, Daxa, DuckDuckGoose and Trocco.

OCT 2021 TO SEP 2025
- WhiteHat Jr

Director Teacher and Mentor, part time

Promoted to Director for quality-managing 500+ live coding sessions. Mentored and supervised a team of instructors.

APR 2019 TO OCT 2021
- Crafter Inc

Intern, Project Coordination and UX

Backlog grooming and delivery tracking for client projects.

Education. B.E. Computer Science, T John Institute of Technology, 2020. UX Design Certification, Springboard, August 2021.

APR TO MAY 2021

120+

CLINICS

50K+

PLATFORM USERS

900K+

TRANSACTIONS YTD

97%

ON-TIME DELIVERY

A patient record is a microsite, not a timeline.

Spry's patient view was a chronological log of everything that ever happened, written in backend language. I audited it, argued that a timeline should not be the front door, checked that call with the people who use it, and rebuilt the record as a case-aware workspace that adapts to the role.

MY ROLE

Product design and ownership, audit to handoff

SURFACE

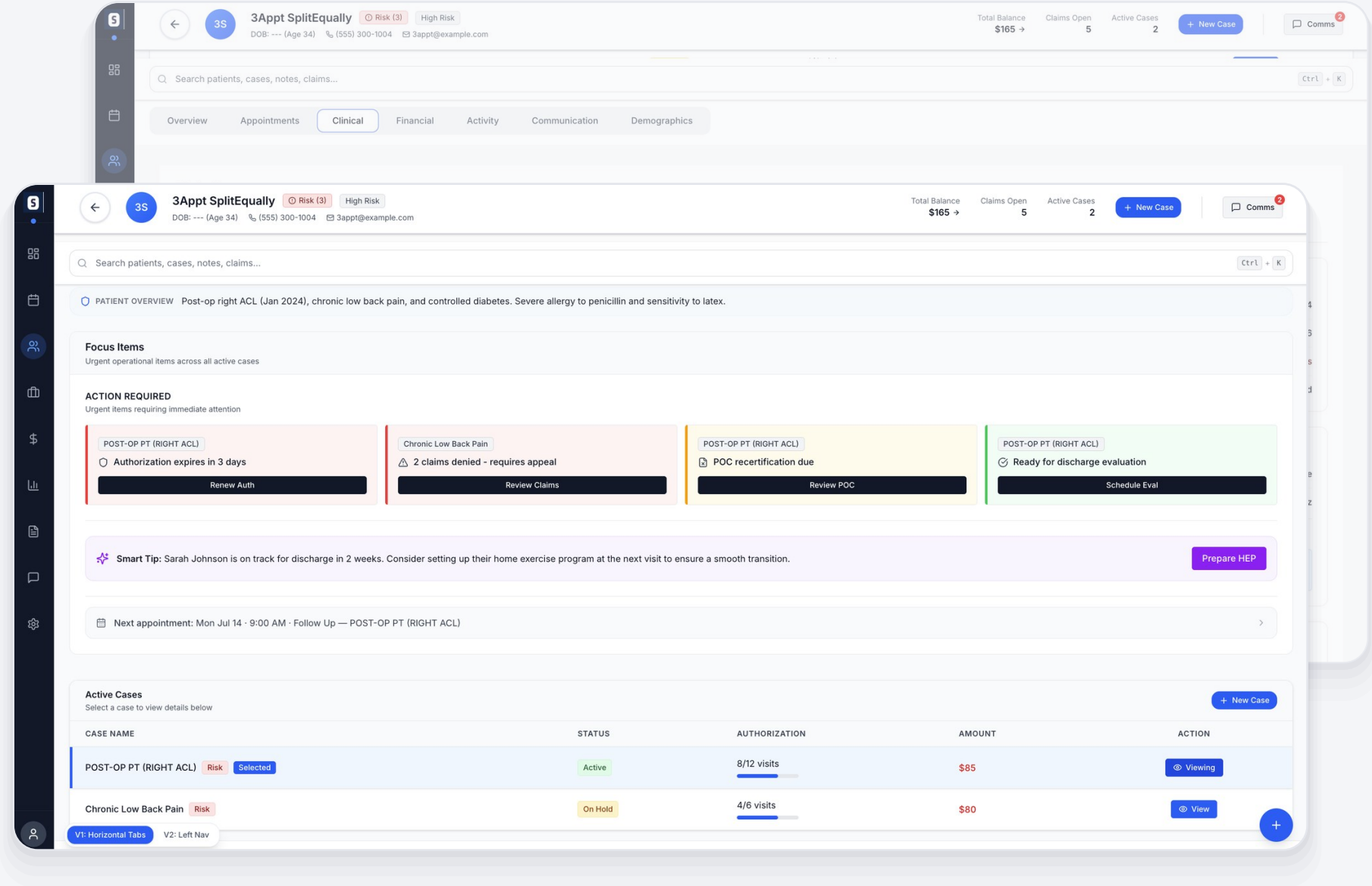
Patient chart: EMR, billing and RCM

CONTRIBUTION

Audit, IA, entity model, PRD, prototype

DELIVERED

Case-first workspace, PRD through Figma handoff



The old timeline described the system, not the patient.

- Payments, refunds and totals looked identical, so you read line by line.
- System words everywhere: "Descending" where a person means "Newest first".
- Events shared a timestamp with no visible order or cause.
- Money, the thing billing cares about most, had no visual priority.
- A heavy static patient panel competed with the task, and nothing planned for history that keeps growing.

If it is a microsite, the timeline is the archive.

A timeline is a comfort pattern. Good for “what happened, when”, bad for “what do I do now”. No amount of polish fixes that, so I changed the model, not the styling.

THE MODEL I MOVED TO

Identity and state
Always pinned. Who, which case, how risky.

What matters today
A short summary and focus items. The primary decision surface.

Timeline | Table | Cards
The archive. Interchangeable views of the same data.

archive

CHECKED WITH FRONT DESK, THERAPIST AND BILLING ROLES

Admin
Wants everything in one place

Therapist
Wants documentation, not claims

Biller
Wants claims and payment status first

I asked about behavior, not layout taste: “What is the first screen a biller opens?” Our design lead's steer was readability over decoration, and curate, do not dump.

Patient first, or case first?

A patient can run several cases at once, say ACL surgery and chronic back pain. Which object the design anchors on decides what you can see, and what you can miss.

OPTION A

Case only

+

Clean and clinically focused

+

Each case reads like its own small chart

-

Hides the patient's total balance

-

A denial in an inactive case can go unseen

OPTION B

Patient anchored

+

Better financial visibility

+

One total balance, one history

-

More complexity on every screen

-

Harder to keep a screen on one episode of care

RESOLUTION

Case first, zoom out on demand

Case view is the default. One explicit toggle, All patient activity, shows the whole patient. Financial offers both a Patient ledger and a Case ledger. I did not remove the tension. I handed it to the user as a control.

Grouped by domain

Timeline entries are grouped Clinical, Insurance and authorization, Billing, each with a visible case tag.

Role changes defaults, not layout

Therapist opens to Clinical, biller to Billing, admin to everything. One workspace, never separate builds.

Which case am I in?

The Overview opens on a Selected case card with an explicit name and status badge.

Some of the missing UI was missing data.

I kept fixing the table until I asked what each row actually represents and which states it can be in. Four things had no real model, so no screen could show them honestly.

Authorization

From "4 of 12 visits" to a real object

- Auth number
- Payer
- Valid through
- Days remaining
- Visits used

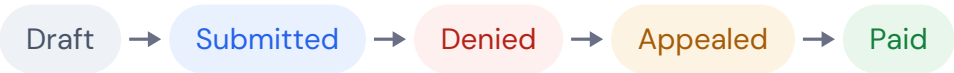
Plan of care

Its own status and lifecycle

- Certified
- Active
- Frequency
- Duration
- Next review

Claim

A real state machine



Ownership

Who is responsible for the fix?

Every denied claim and expiring authorization gets an assignee, so nothing sits in a shared queue with no name on it.

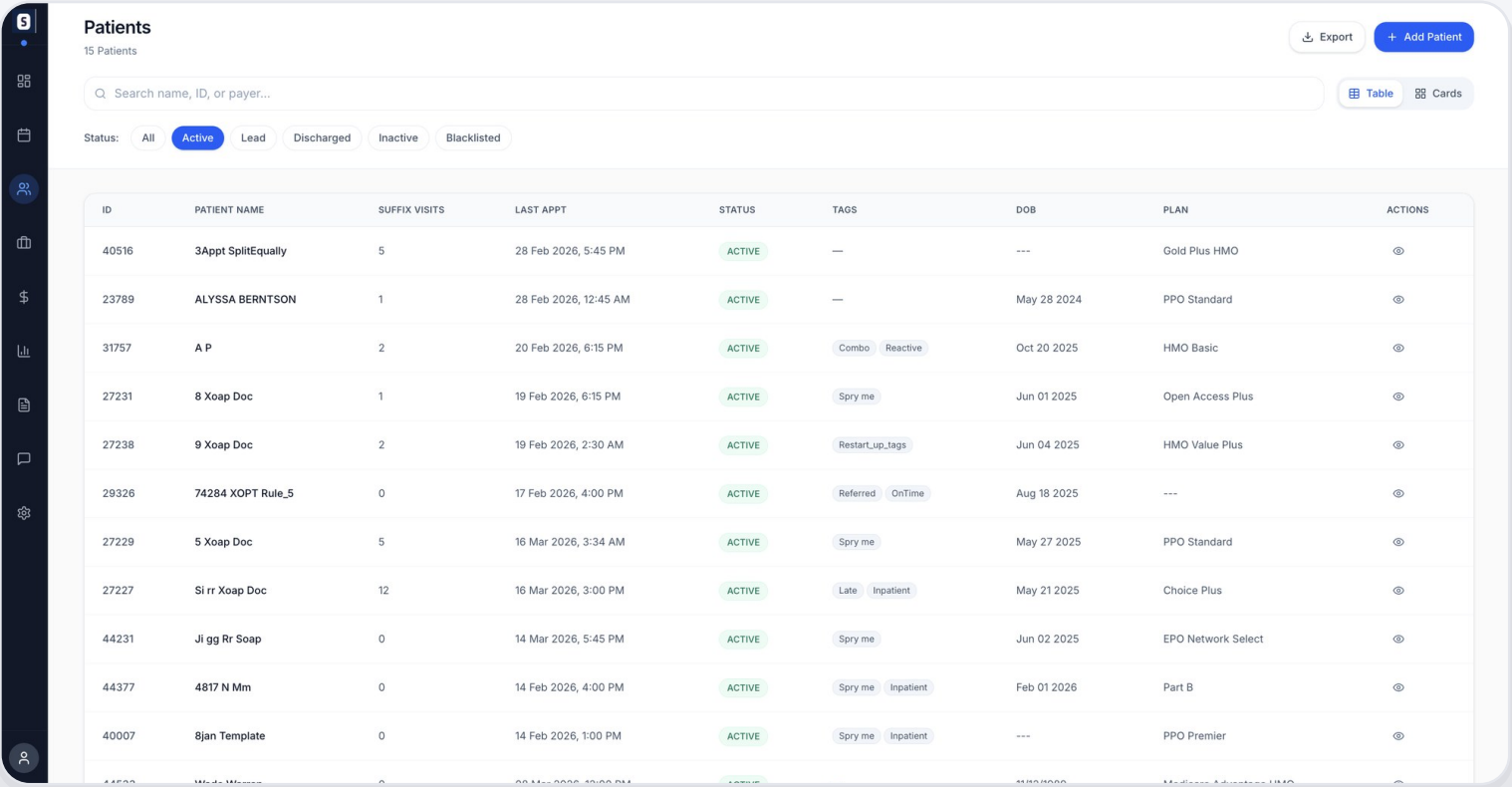
Severity on top of all of it

One shared vocabulary for how loud an alert should be.

- Critical
- Warning
- Info

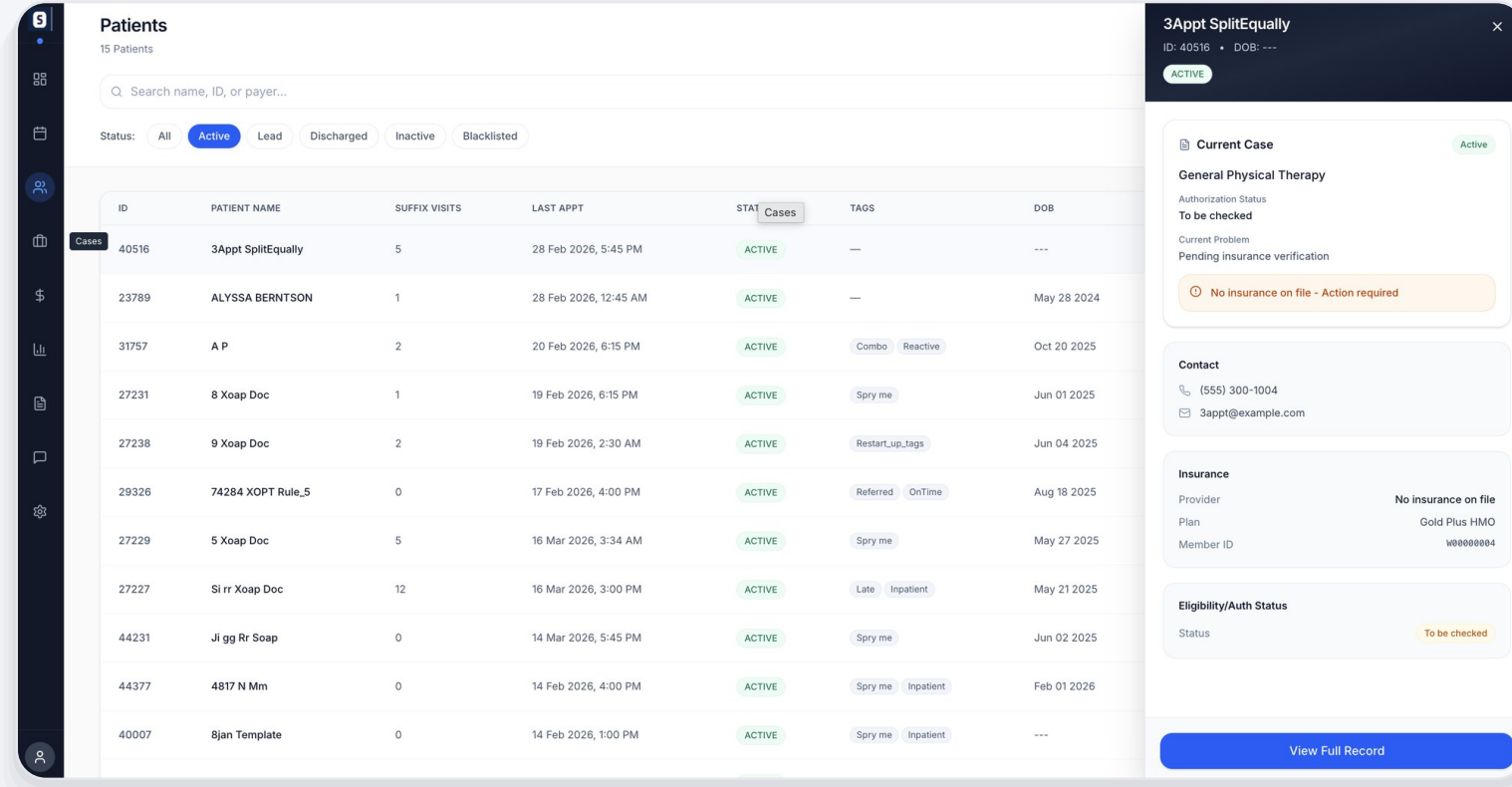
These are not decorations. They are the direct result of asking: what object is this, and what state can it be in?

Peek first. Open the record when you need it.



1 The patient list

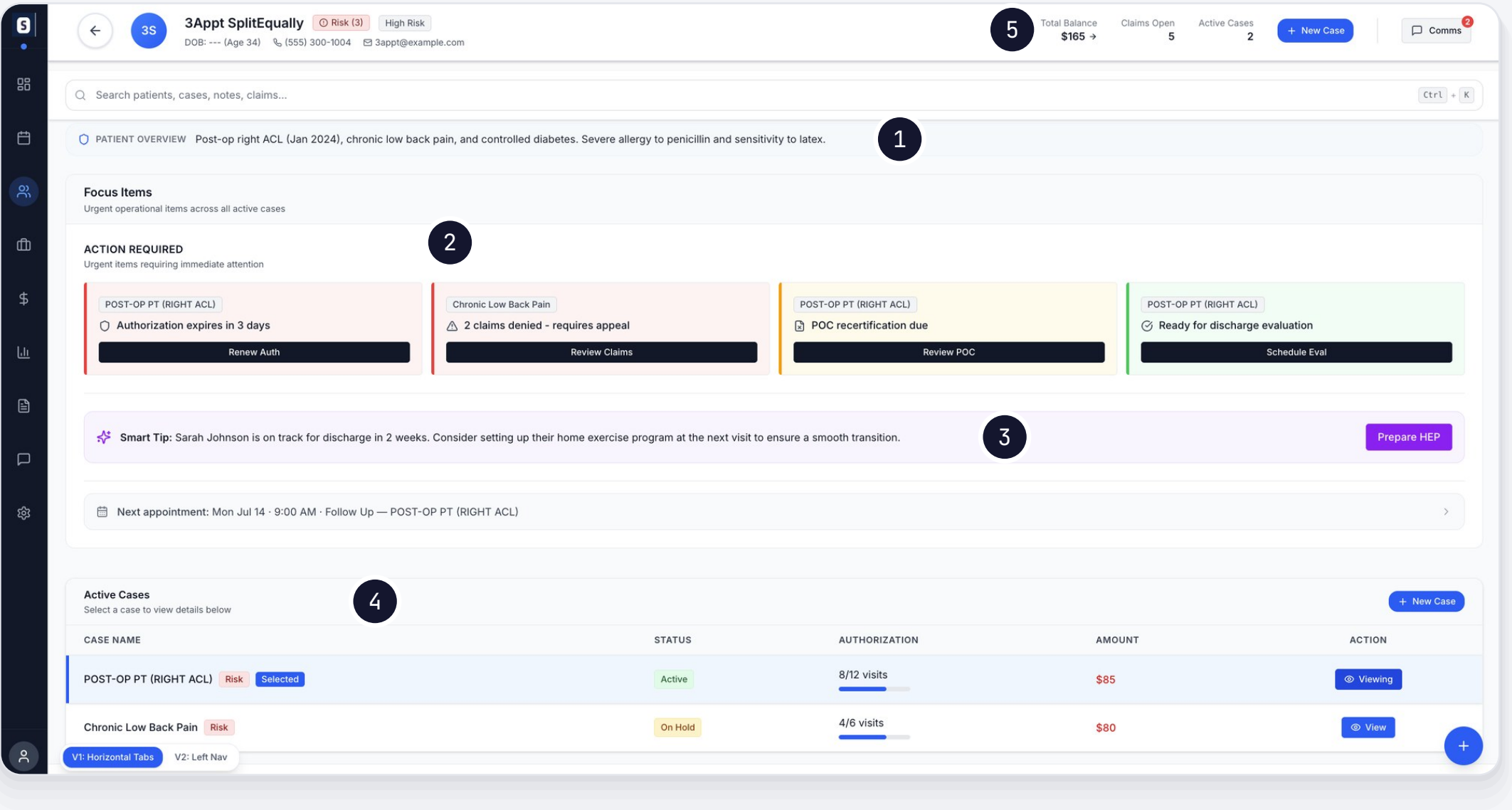
Search by name, ID or payer, status filters, and a Table or Cards view. Built for finding someone fast, not for reading everything.



2 The quick-look drawer

A row click opens a drawer, never a new page. It shows the current case, its authorization status, the one problem blocking it ("No insurance on file, action required"), then contact, insurance and eligibility. View Full Record opens the workspace.

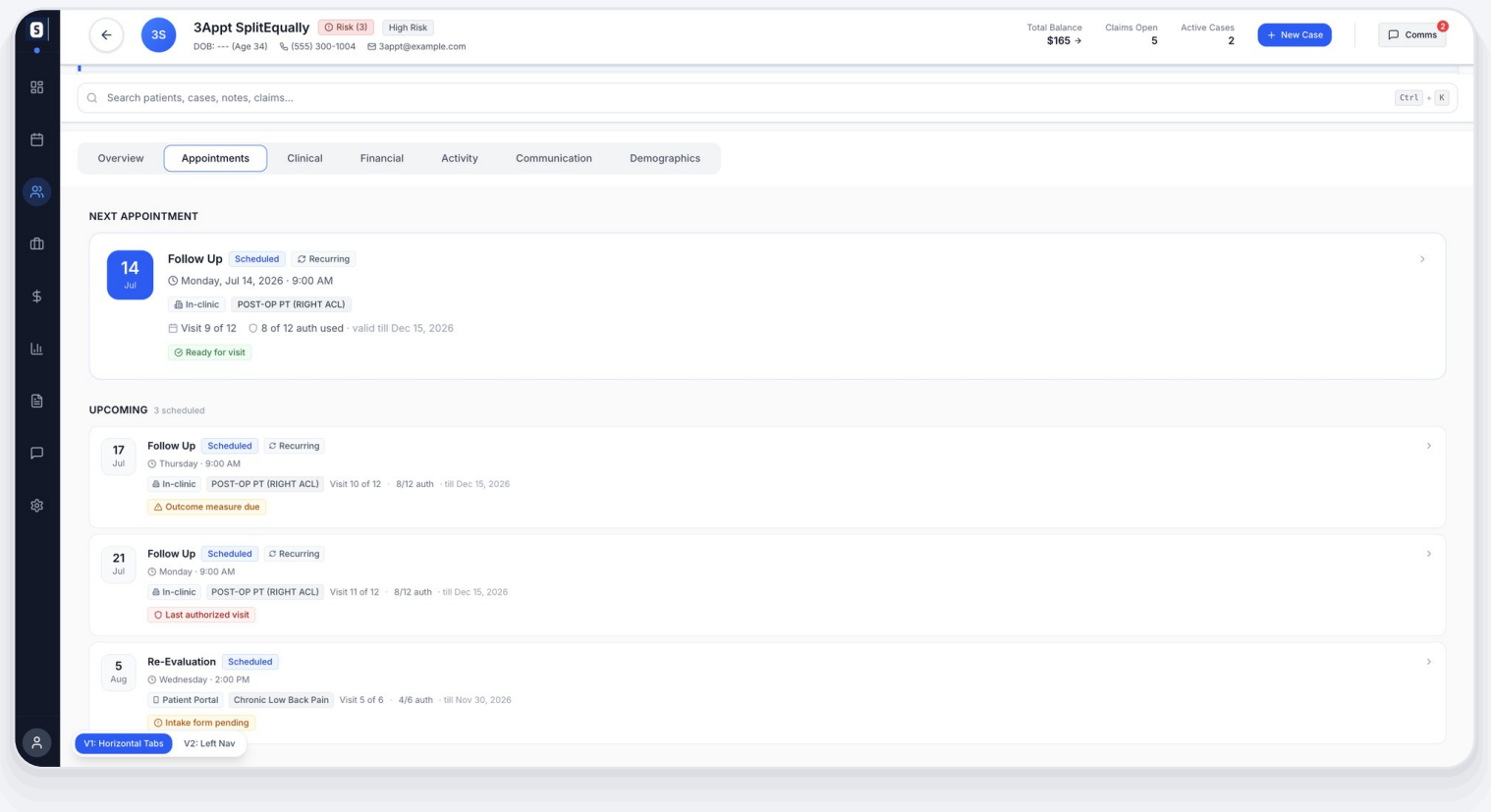
Open a patient and see what needs you today.



WHAT TO NOTICE

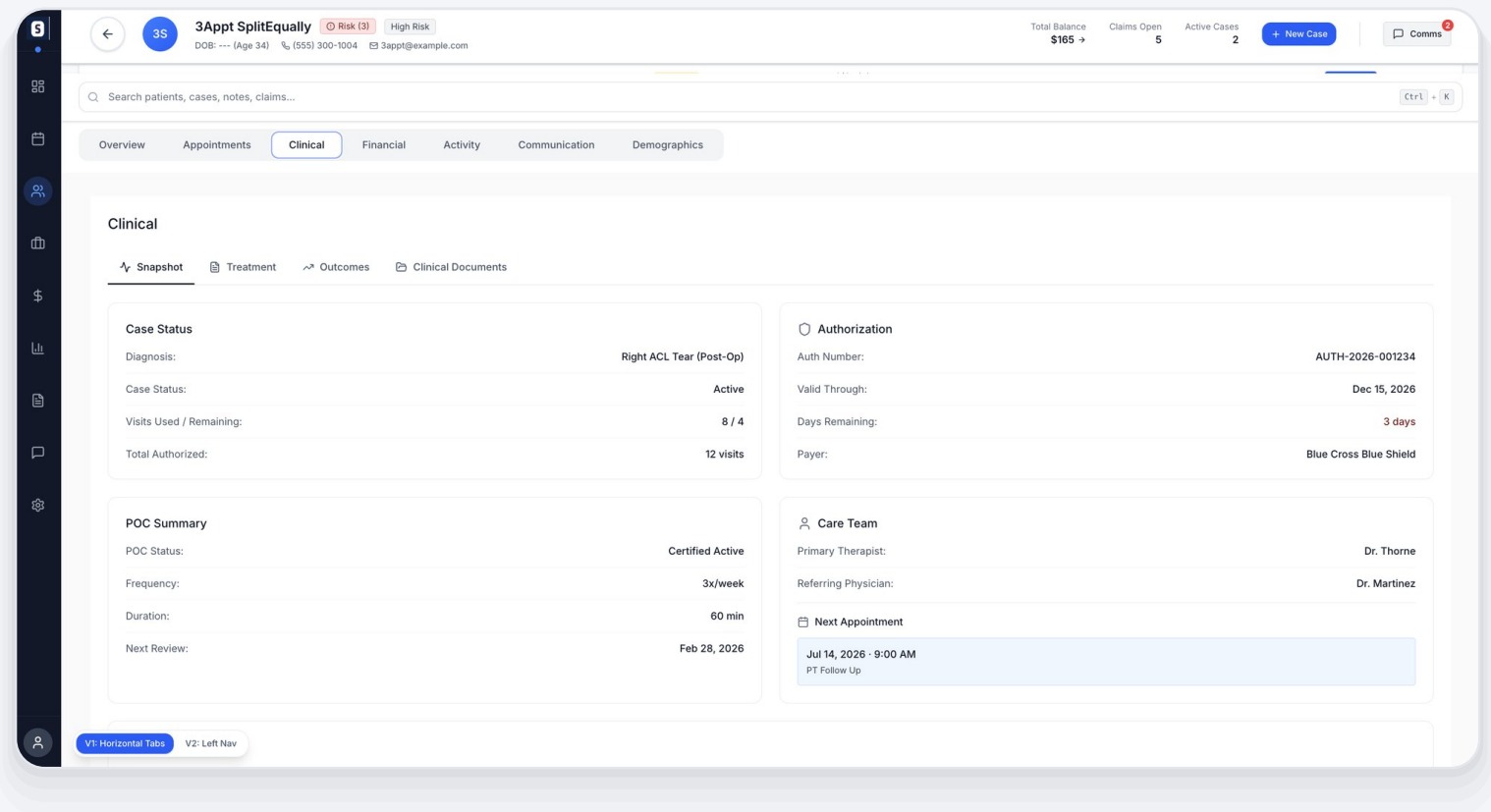
- 1 One-line patient overview**
Diagnoses, allergies and context in a sentence, so it is never behind a click.
- 2 Action required, by severity**
Cards are colored red, amber, green, tagged with their case, and each carries one button: Renew Auth, Review Claims, Review POC, Schedule Eval.
- 3 Smart tip, kept apart**
The AI nudge sits in its own purple band, so suggestions never get mistaken for required work.
- 4 Active cases, explicitly**
Selected and Viewing tell you which case you are in, with authorization progress and the amount per case.
- 5 Always-on totals**
Balance, open claims and active cases stay in the header whichever tab you are on.

Each tab answers one question.



1 Appointments

Started from a customer ask: see a patient's appointments at a glance, without exporting to Excel or scrolling the timeline. Next appointment first, then upcoming, each row with its case, visit number and auth use ("Visit 9 of 12, 8 of 12 auth used"), plus flags like Last authorized visit.



2 Clinical

Case status, Authorization with a days-remaining countdown (3 days, in red), POC summary and Care team as separate object cards. The next appointment lives inside the care team card.

Overview What needs me today	Appointments When are they in	Clinical Where the care stands	Financial Patient or case ledger	Activity The archive, filterable	Communication Who said what	Demographics Patient shell, never case-scoped
--	---	--	--	--	---------------------------------------	---

One billing product that changes shape for seven kinds of practice.

Spry sells software and services in different combinations. A clinic might run everything itself, hand claims to Spry's RCM team, or buy nothing but eligibility checks. Instead of seven separate builds, I designed one configuration-driven system that reshapes itself around what each practice actually owns, and recommends actions with AI while routing the uncertain ones to a human reviewer.

MY ROLE

Product design and ownership, end to end

SURFACE

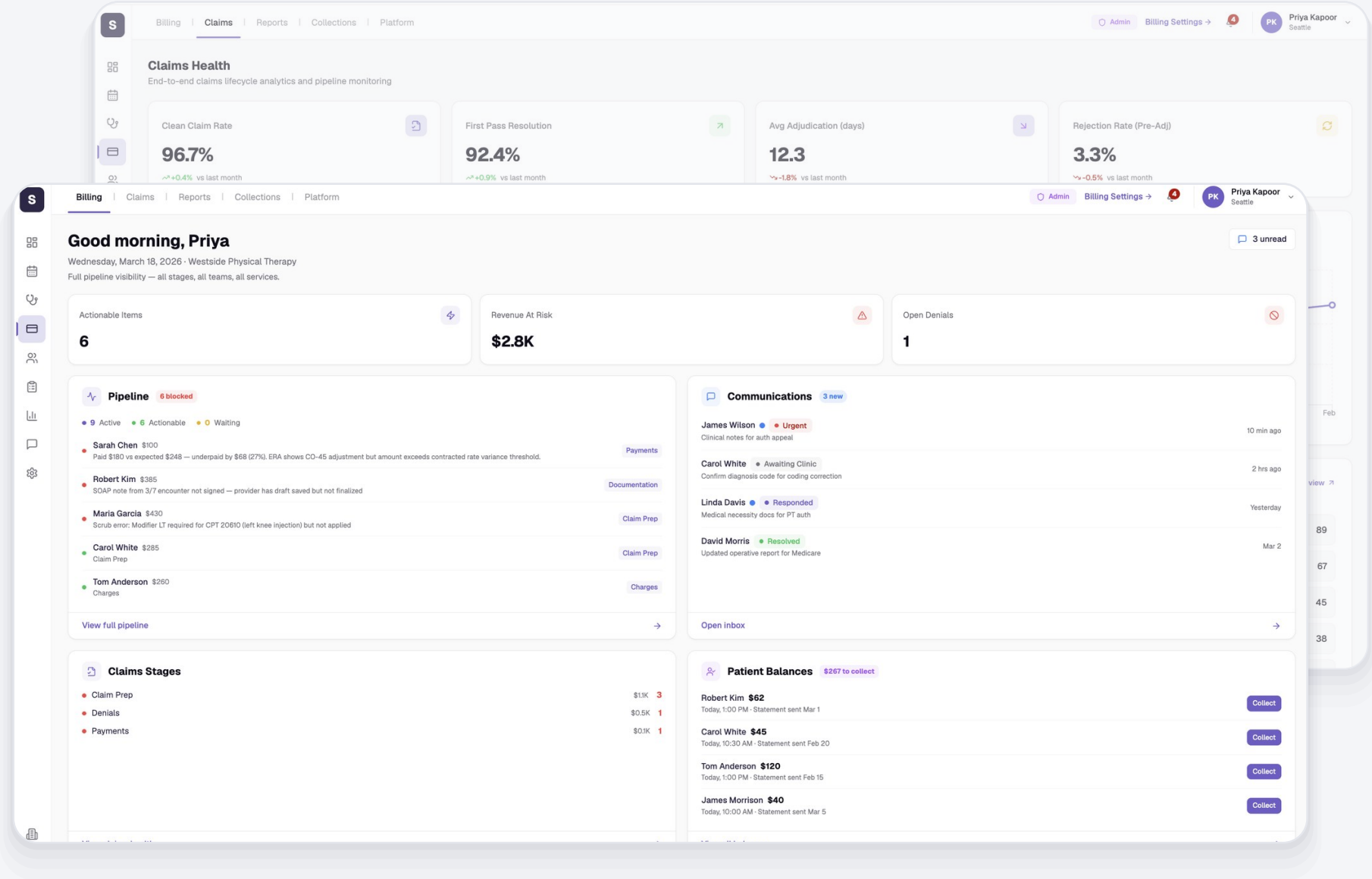
Unified Billing Dashboard

PROCESS

15+ major iterations, discovery to engineering delivery

DELIVERED

Bracket model and layout rules for every configuration



Seven ways to sell one product. So the sidebar is the bracket.

THE PROBLEM

- Software varies. Spry EMR or an outside one. Spry billing or none.
- Services vary. RCM, eligibility and prior auth, or neither.
- Ownership varies. The clinic's biller or Spry's team does the work. Same screen, different meaning.
- Complexity is real. The fullest build is dense. The simplest must not feel like a leftover.

Seven separate builds drift apart. One screen with unused parts greyed out makes everyone feel they pay for what they cannot use. So each clinic maps to a bracket, B1 to B7, defined on three axes:

Ownership	who executes: the clinic's biller or Spry's team
Mode	execute, collaborate or observe
State	where each item sits, from charge to payment

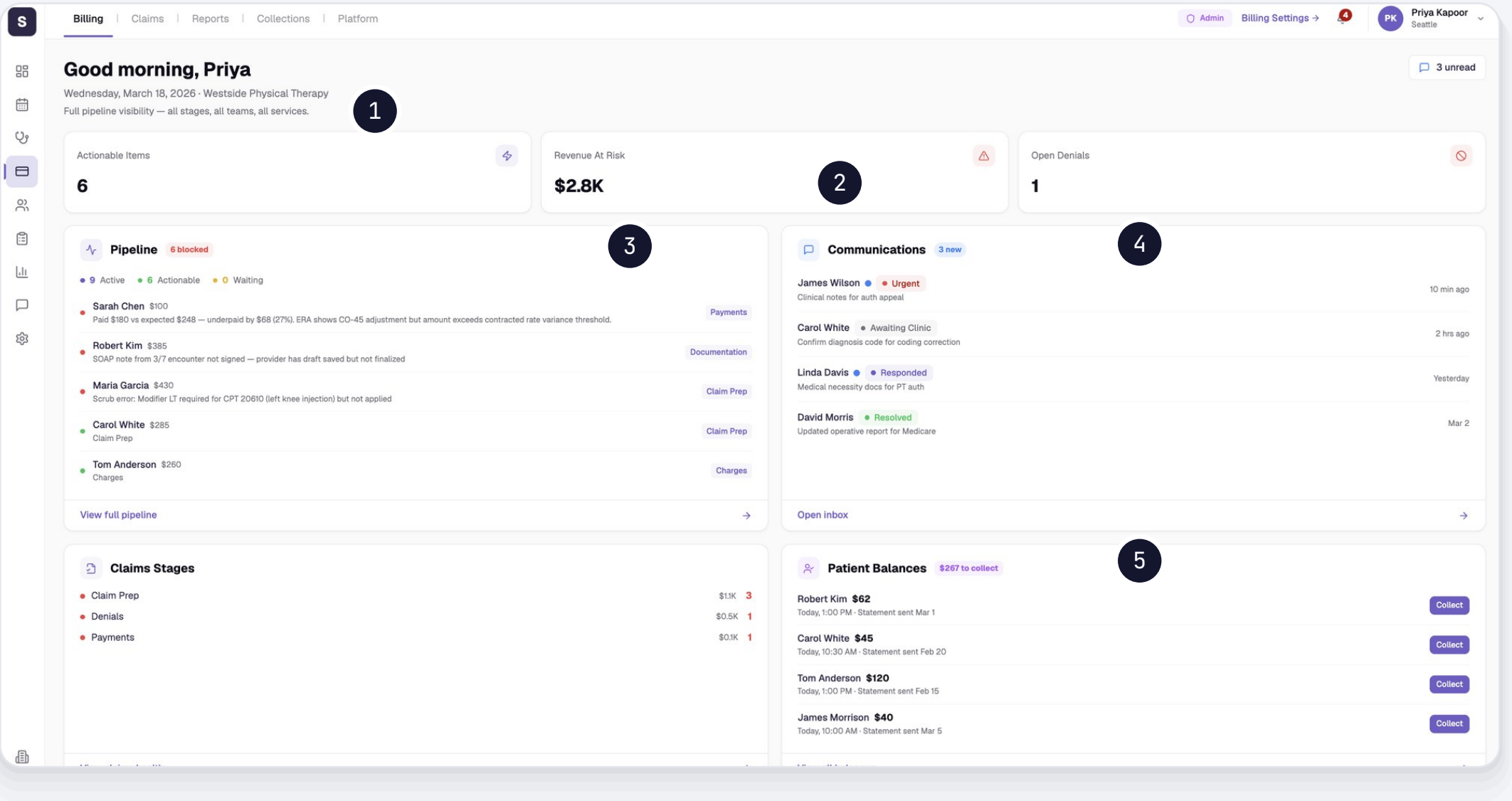
If it is not in your configuration, it does not exist in your nav.

ONE DENIED CHARGE, FOUR CONFIGURATIONS

	B1 Software only	B2 + RCM service	B3 Full stack	B7 Eligibility only
Denied claim	Work denial	• Spry RCM	• Spry RCM	No charges
Prior auth	Request auth	Request auth	• Spry Auth	Verify queue
Left rail	Clinical, Billing	+ Chat	+ Chat	Verify only
Biller sees	Buttons	Status, some buttons	Mostly status	Its own product

Status is always visible. Buttons only appear for work you own.

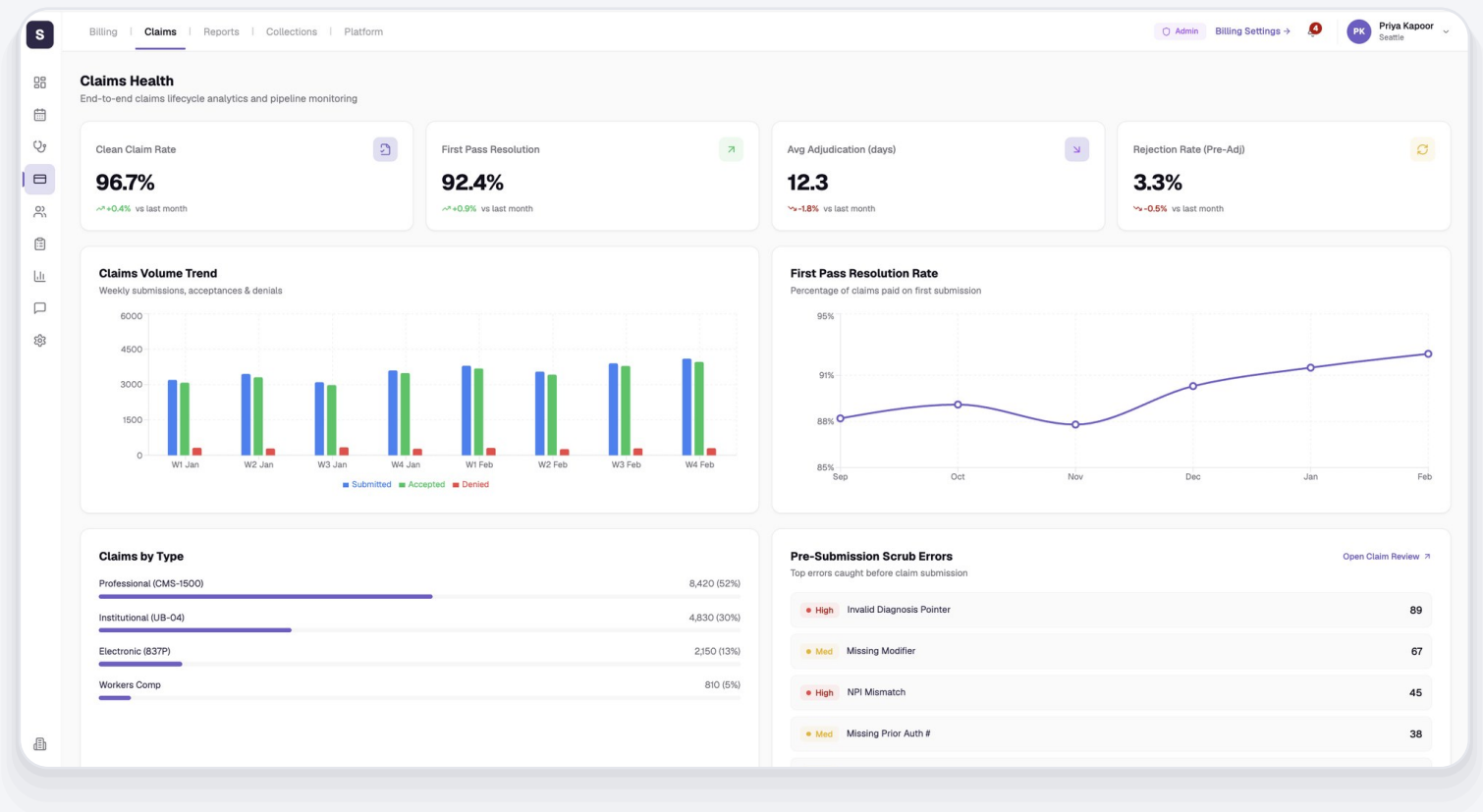
A task queue, not a dashboard.



WHAT TO NOTICE

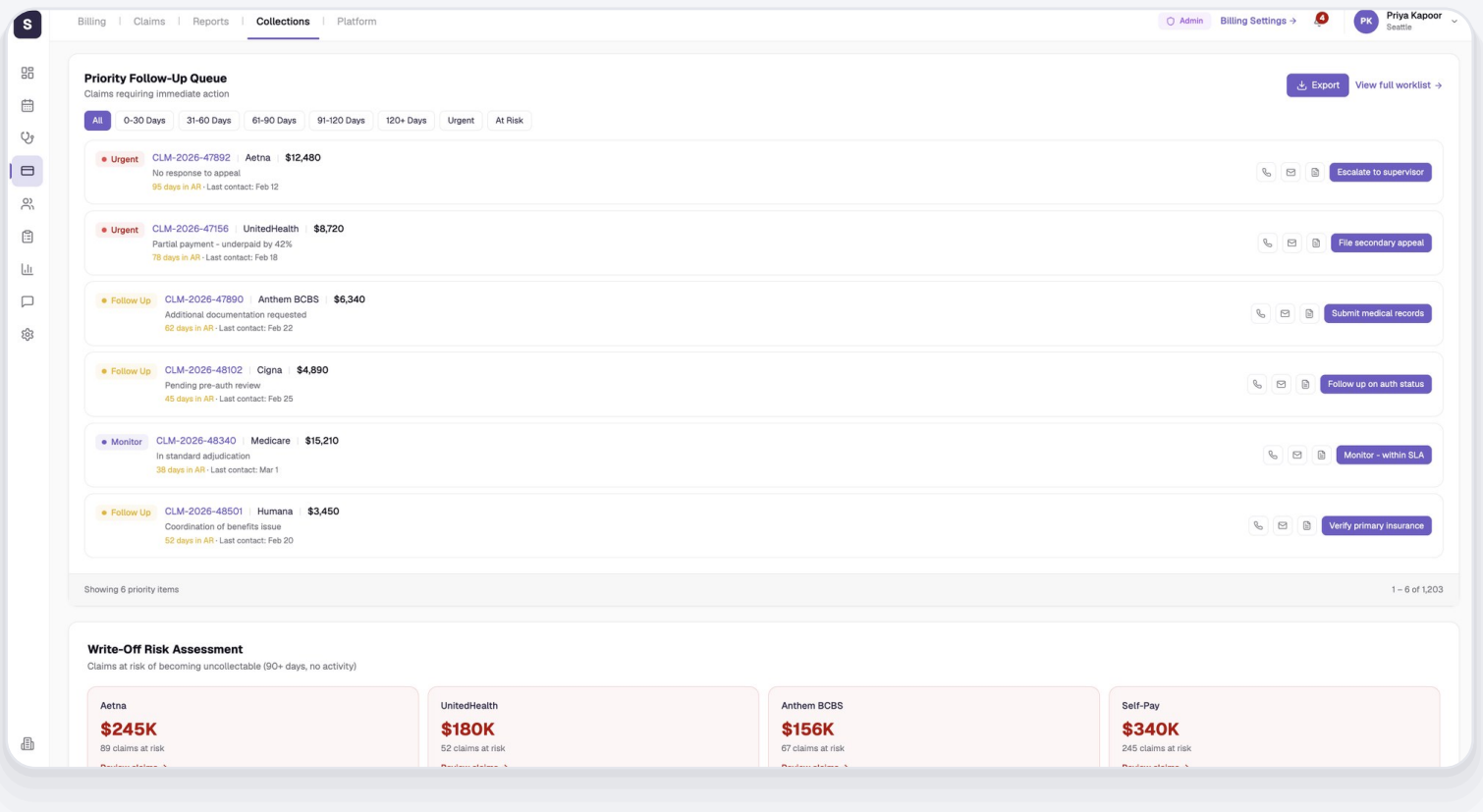
- 1 Scope, stated up front
"Full pipeline visibility: all stages, all teams, all services." The page says which configuration you are in.
- 2 Three numbers, no more
Actionable items, revenue at risk, open denials. Everything else earns its place lower down.
- 3 Blocked work explains itself
Each item says why it is stuck in plain language, with its stage: Payments, Documentation, Claim Prep, Charges.
- 4 Conversations with state
Urgent, Awaiting Clinic, Responded, Resolved, so the inbox sorts itself by who owes the next move.
- 5 Money to collect, one tap away
Patient balances carry a Collect button on every row.

Analytics where it helps. Work queues where it counts.



1 Claims Health

End-to-end claims analytics: clean claim rate, first pass resolution, adjudication time and rejection rate, each with its change against last month, plus the top pre-submission scrub errors ranked by severity.



2 Collections

A priority follow-up queue sorted Urgent, Follow Up, Monitor, with age filters and the next best action on every row: Escalate to supervisor, File secondary appeal, Submit medical records, Follow up on auth status, Monitor - within SLA, Verify primary insurance.

Billing

What needs doing today

Claims

How healthy the pipeline is

Reports

The long view

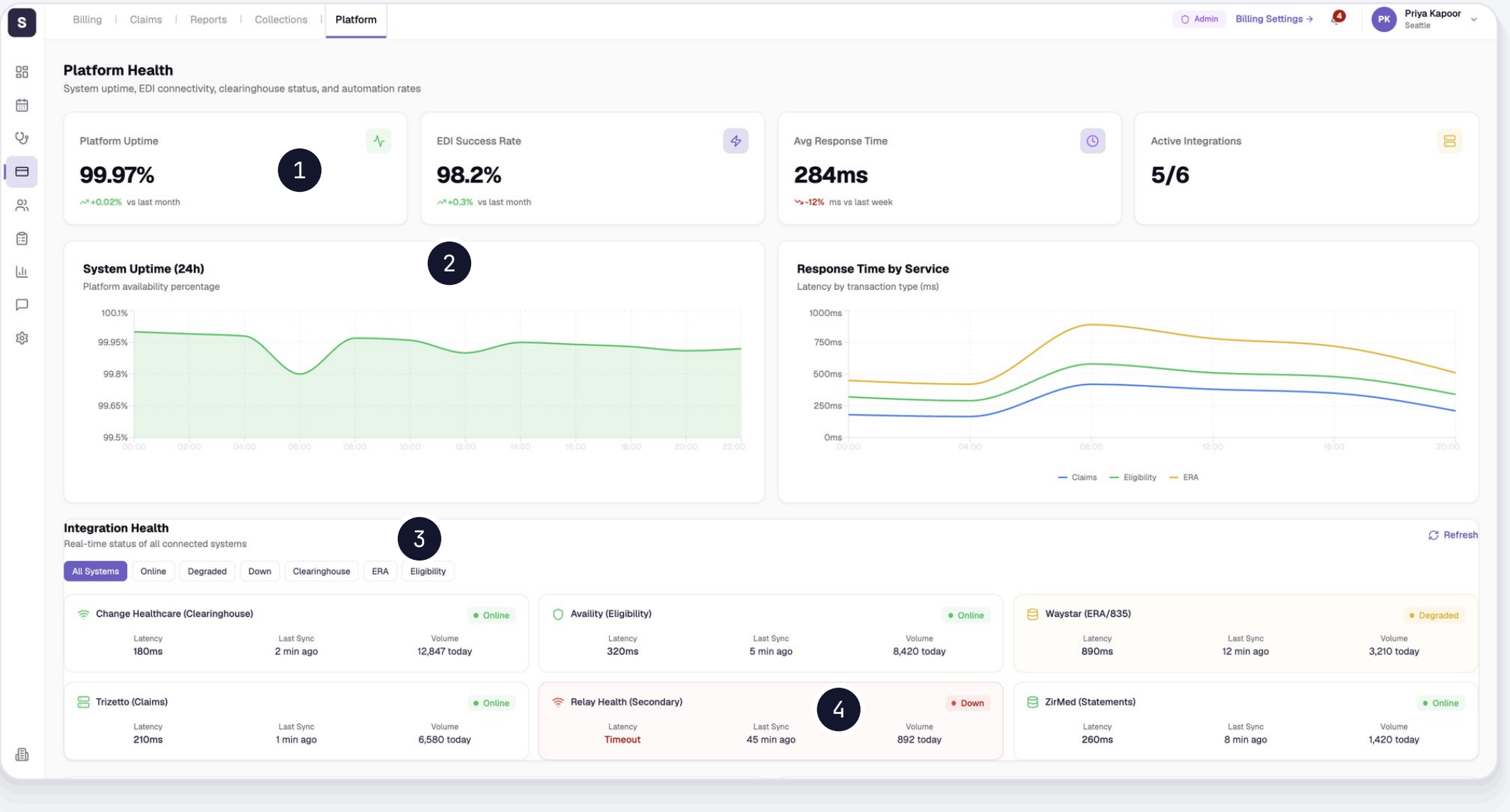
Collections

Who to chase, in what order

Platform

Whether the plumbing is up

When a connection fails, say so before the numbers do.



WHAT TO NOTICE

- 1 System health at a glance
Uptime, EDI success rate, response time and active integrations, each against the previous period.
- 2 Trends, not just totals
Twenty-four hour uptime and response time by service: claims, eligibility, ERA.
- 3 Every connection, filterable
Clearinghouse, eligibility, ERA and statements, with filters for Online, Degraded and Down.
- 4 Failures read first
Down and degraded systems are tinted, so a stuck claim is not mistaken for a payer problem.

Where verification coordinators start the day.

SPRY Verify has to answer one question fast: what do I act on, and in what order? It brings real-time eligibility checks, prior auth tracking and denial management together for front desk and admin teams, syncs appointments from Epic MyChart, and falls back to an Excel upload when it has to.

MY ROLE

Product design and React build

PROCESS

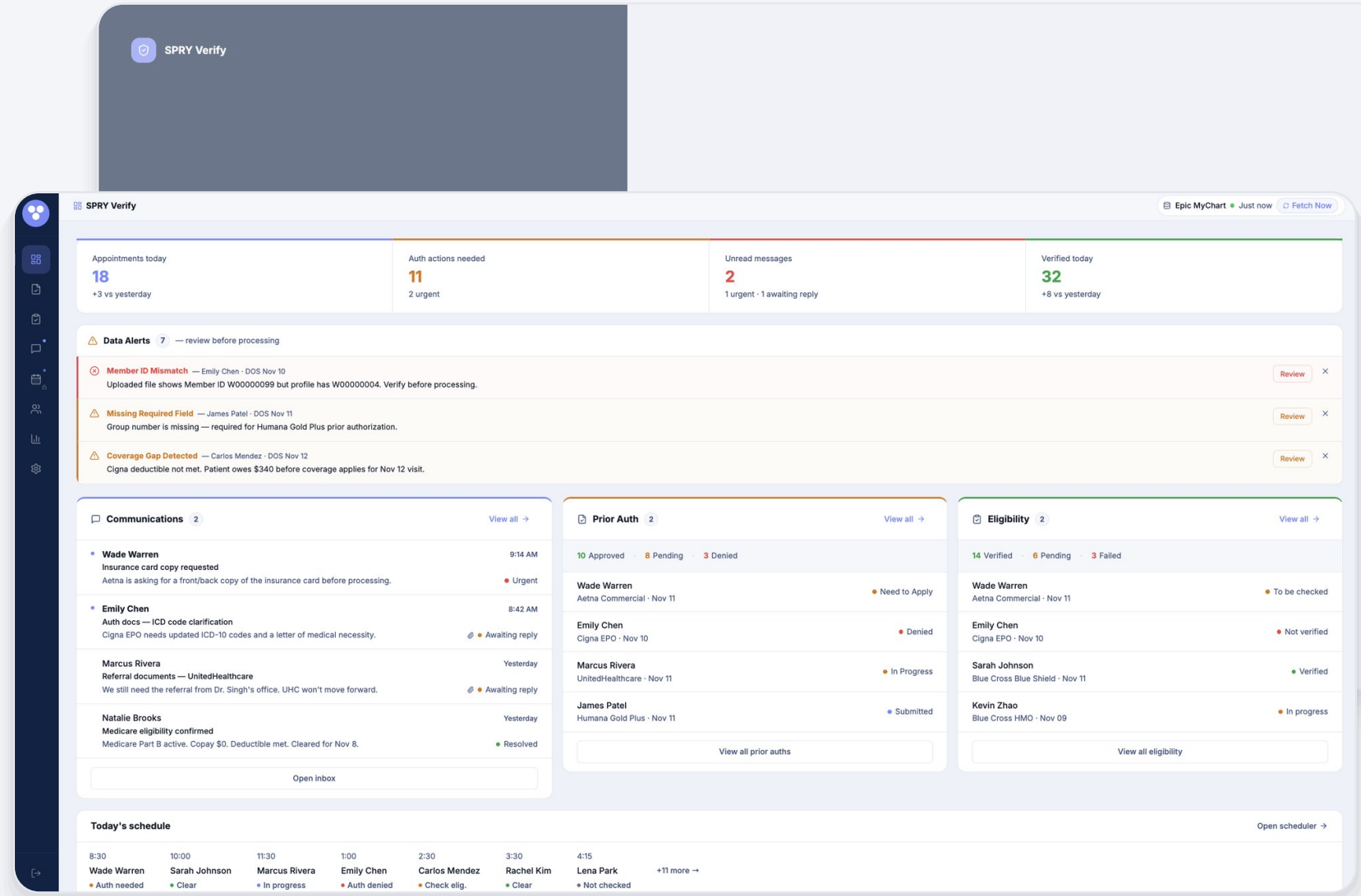
Seven iterations, shaped by operator research

STACK

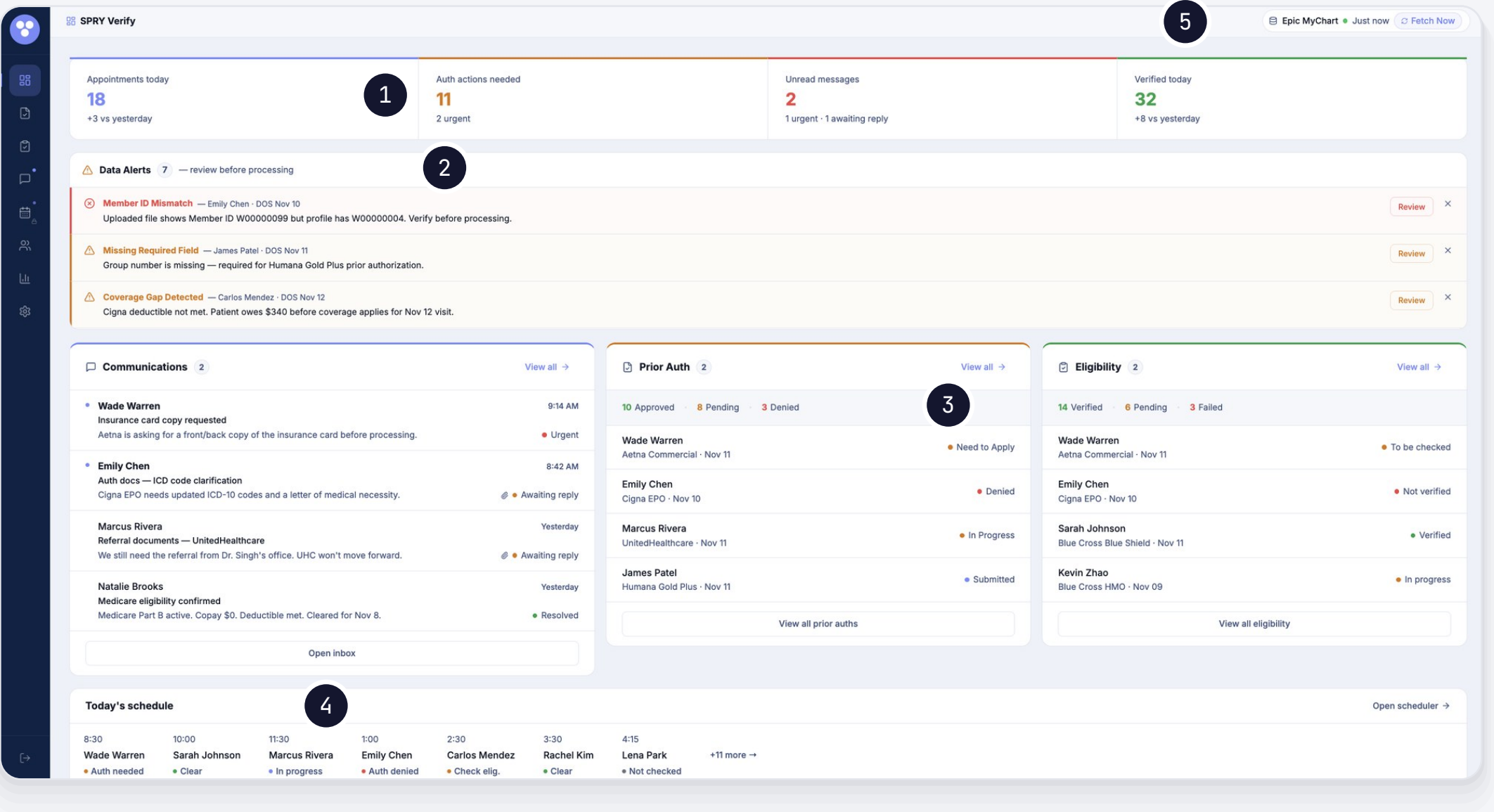
React Router, Tailwind, shadcn, Epic MyChart sync

PRINCIPLE

Task-based, not info-display



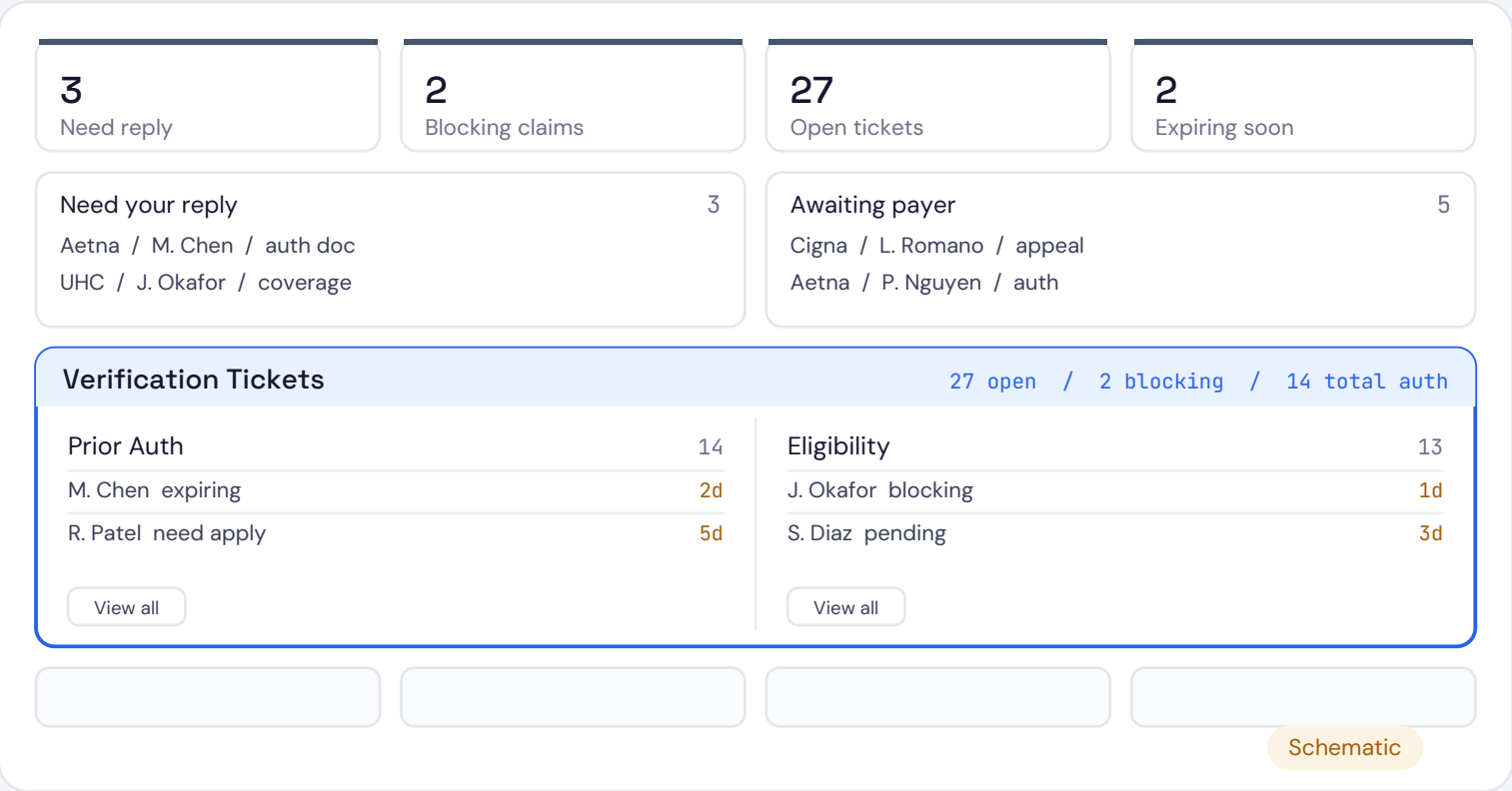
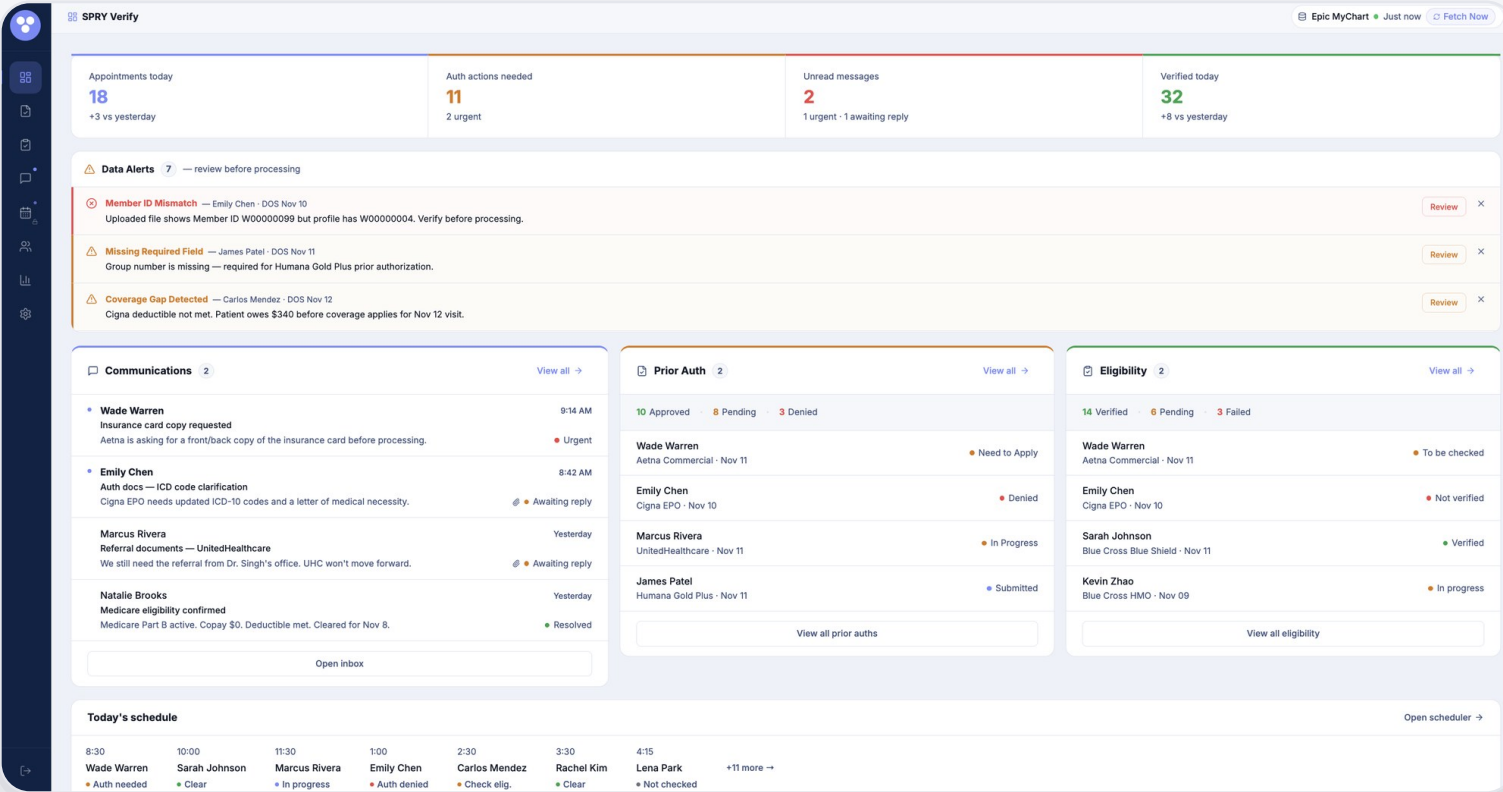
Numbers, then problems, then the work, then the schedule.



WHAT TO NOTICE

- 1 Four numbers, four colors
Appointments today, auth actions needed (11, 2 urgent), unread messages, verified today. Blue, amber, red, green carry the meaning.
- 2 Data alerts above the work
Mismatches and gaps are flagged before anyone processes a record, each with a Review button and a dismiss.
- 3 Counts before names
Every panel leads with its status totals, so you know the shape of the queue before you read a row.
- 4 Today's schedule
Every appointment with its verification status: Auth needed, Clear, In progress, Auth denied, Check elig, Not checked.
- 5 Sync you can see
Epic MyChart status and a Fetch Now control stay in the corner, always.

The dashboard answered the same question twice.

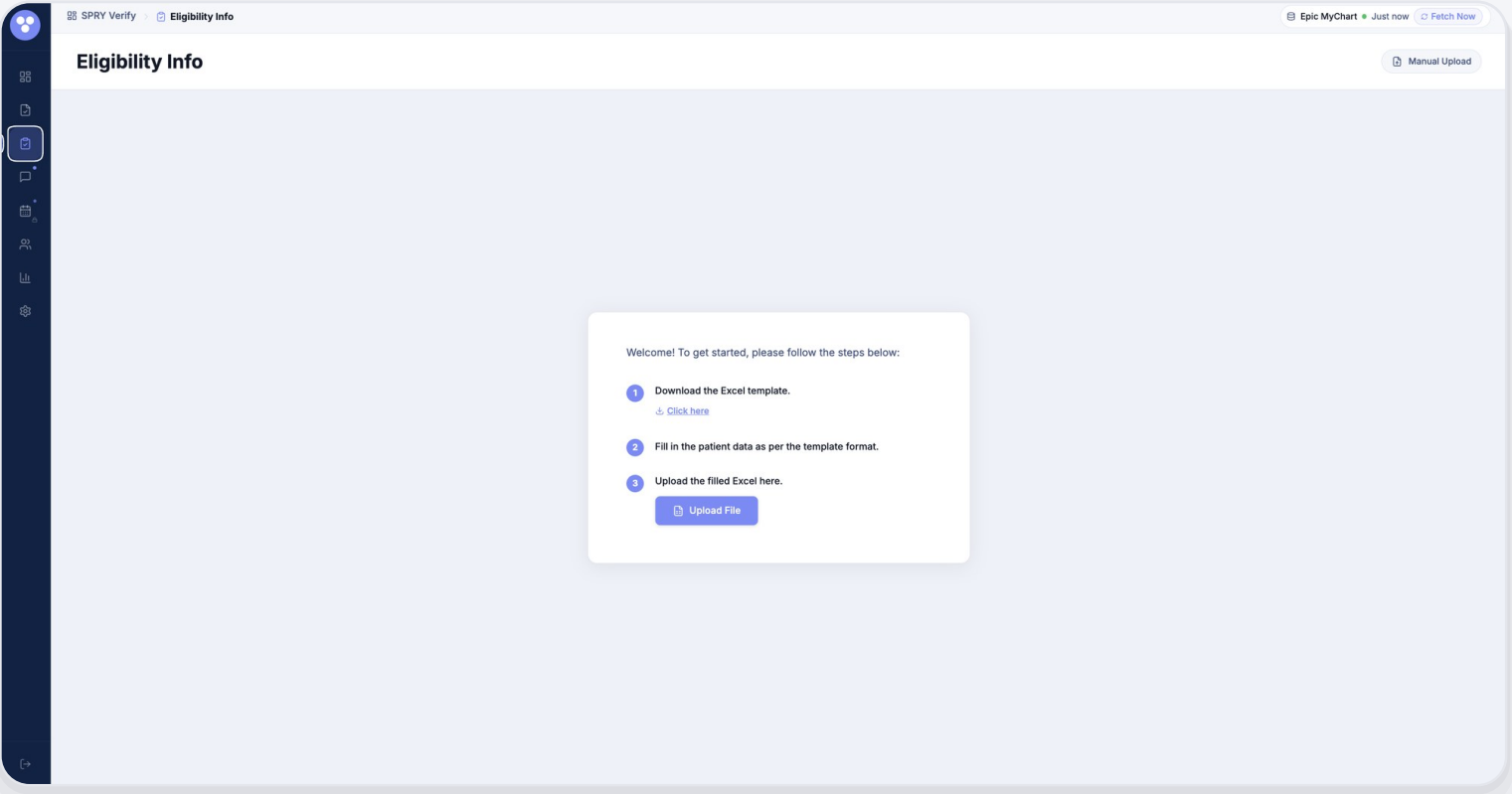


A Stage A / Orient, then act
Four colored numbers, Data Alerts, three panels that lead with counts, and today's schedule. It tells someone where they stand in seconds. Its limit: the three panels read at similar weight.

B Stage B / Group by job, rank by age
Prior auth and eligibility sit in one Verification Tickets container with a shared count strip. Communications shrink to what needs a reply, and every ticket row carries its age. Redrawn here as a schematic.

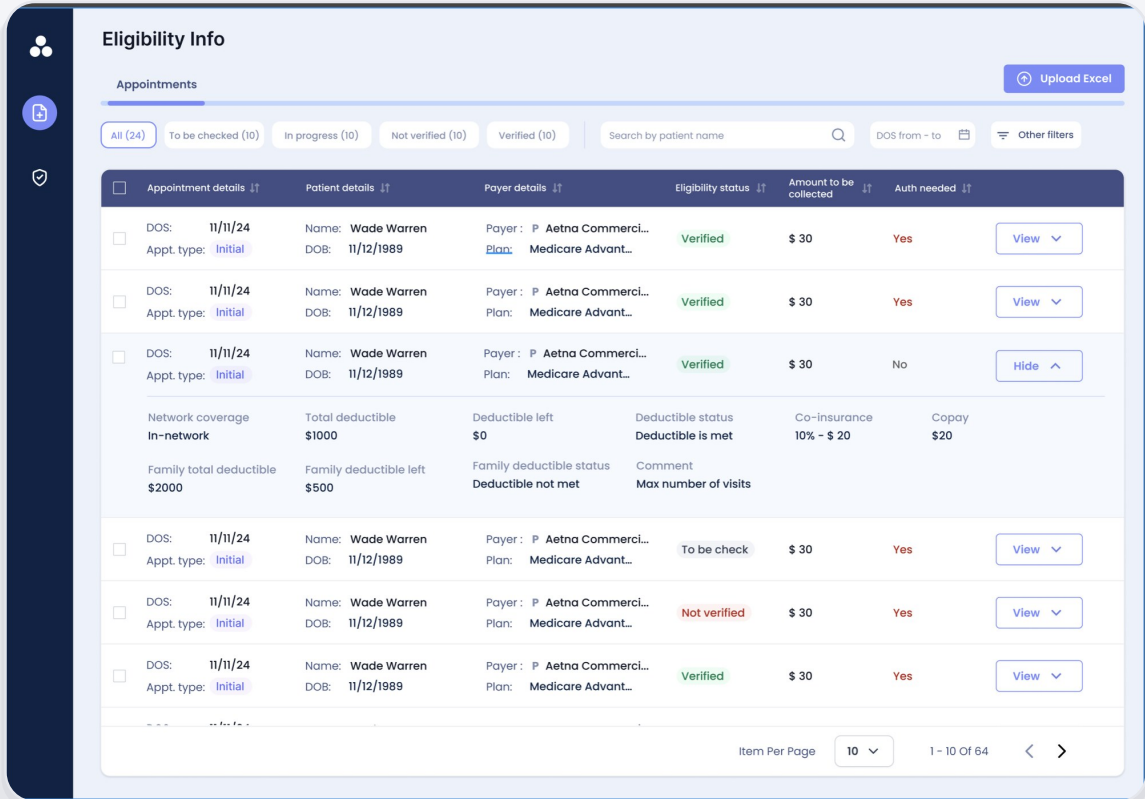
What carried across both stages: task-based, not info-display. Counts first, one next step on every element, and a stress test of 50+ urgent messages and 200+ tickets.

From the first upload to a verified list.



1 Getting started

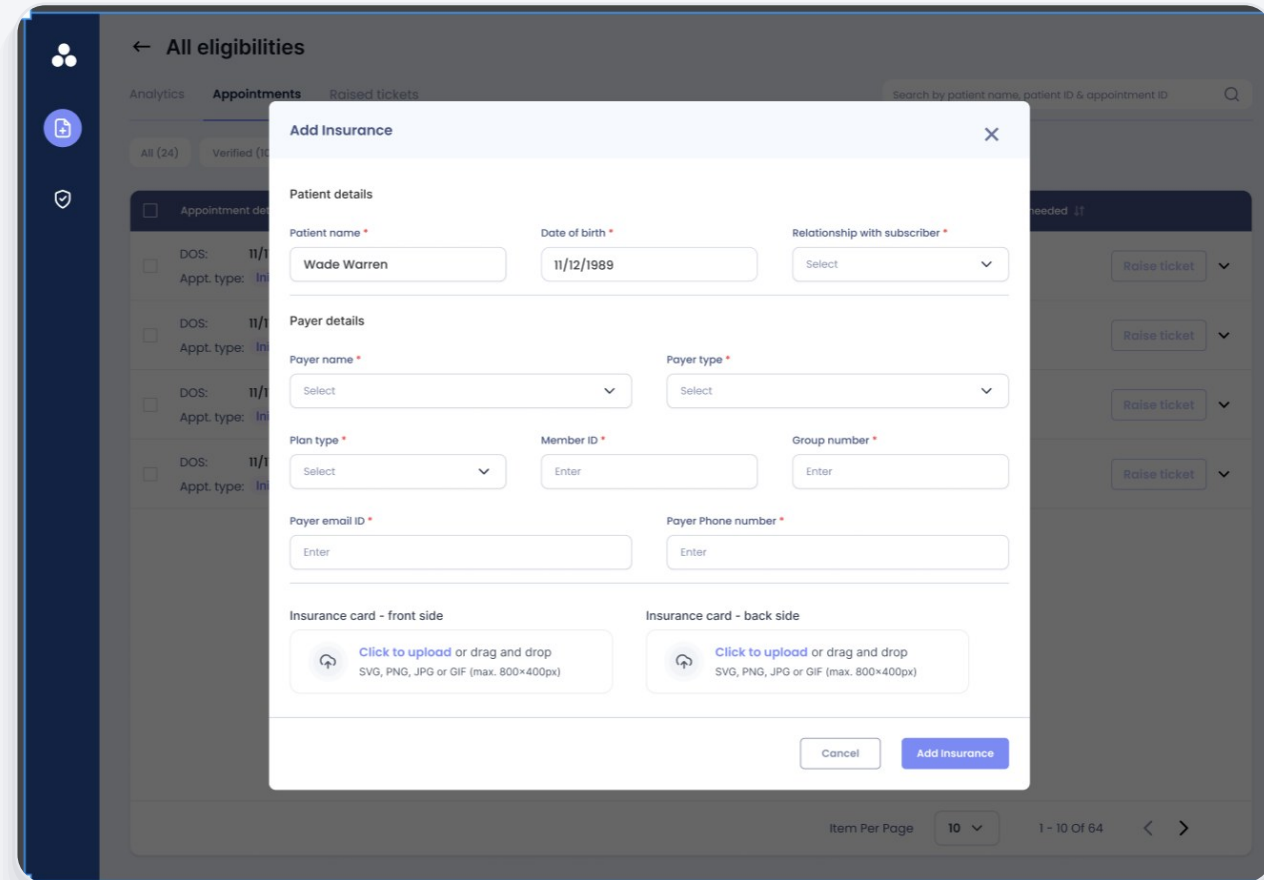
The empty state teaches the whole workflow in three steps: download the Excel template, fill in the patient data, upload it. No manual, no tooltip tour.



2 The eligibility list

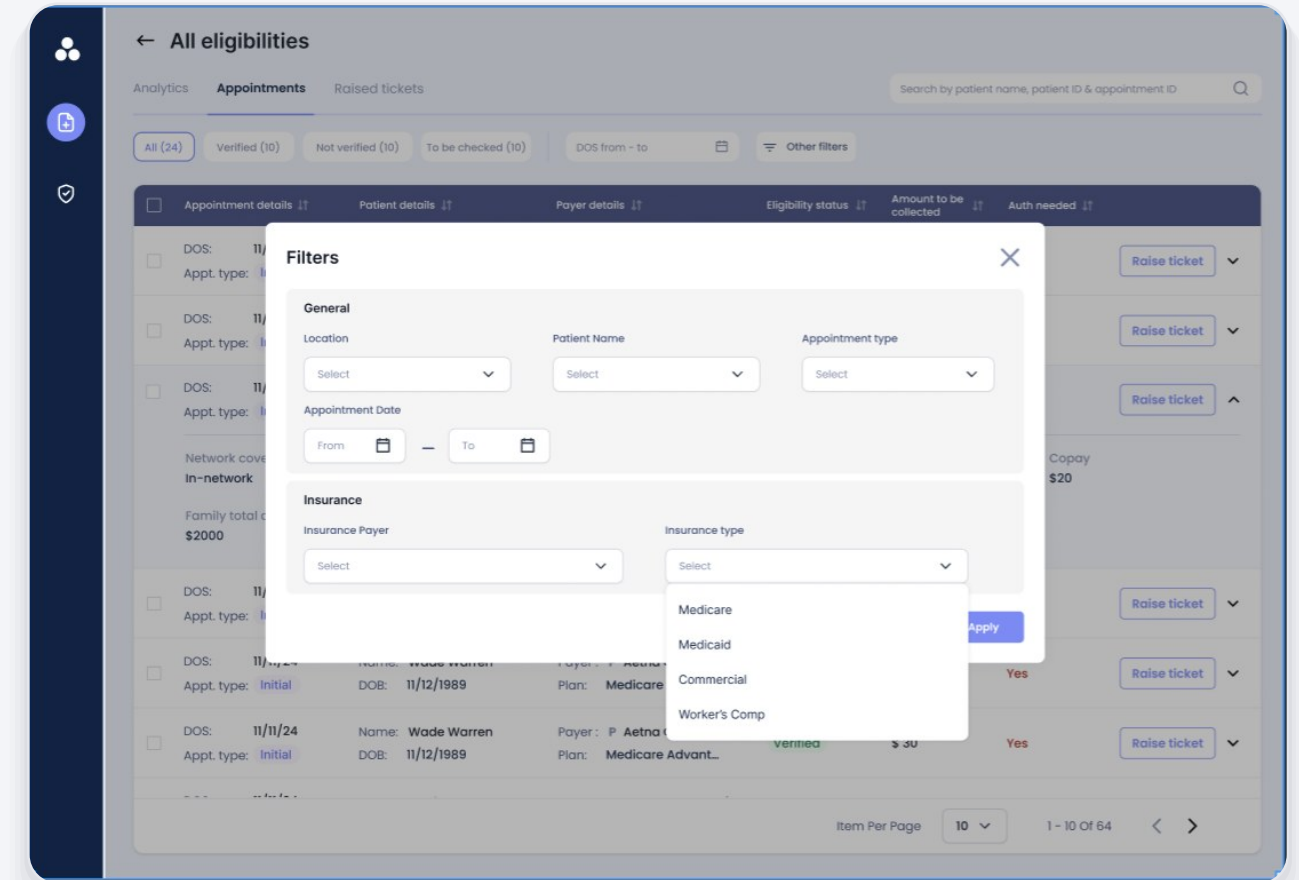
Status filters that always show counts: All 24, To be checked 10, In progress 10, Not verified 10, Verified 10. Each row carries what the front desk needs: payer, plan, amount to collect, auth needed. Expand a row for deductible, co-insurance, copay and network.

The small screens that keep people in the list.



1 Add insurance

When coverage is missing, one modal holds it all: patient, payer, plan, member and group ID, and both sides of the insurance card. Required fields are starred, and you never leave the list.



2 Filters

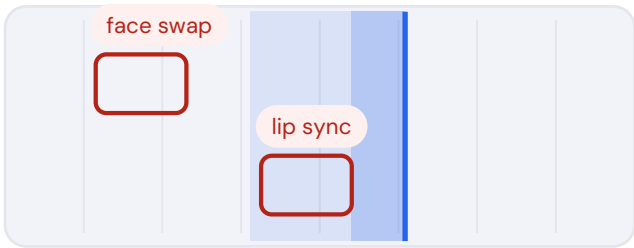
Location, patient, appointment type and date range, plus payer and insurance type: Medicare, Medicaid, Commercial, Worker's Comp. Filters live in a modal so the list keeps its full width.

50+ websites, one habit: find the idea that makes it click.

CLIENTS INCLUDE Agilisium Apex Covantage Daxa DuckDuckGoose Trocco and others

Designing trust for deepfake detection

DuckDuckGoose, via Nitrous / UX Designer



Sweeps the file, finds the anomaly, names its type.

WHAT I DID

Scanned Hive, Reality Defender, Hyperverge, Sensity.ai and Microsoft Video Authenticator, and went through the model logic with the product team. Most tools were too opaque or too technical. The idea: a radar that sweeps a file and names the anomaly, in a calm tone with no AI hype.

OUTCOME

Prospects in media, legal and enterprise security could see how it works and why to trust it.

[Read the full case study on Framer](#)

Turning data workflows into human-centered experiences

Trocco, via Nitrous / UX Strategist and Messaging Architect



WHAT I DID

Audited Fivetran, Riverty, Airbyte and Hevo Data and ran workshops with the product team. The idea: an automated conveyor belt from ingestion to transformation to output, which drove the IA and the wording. Messaging mapped by persona, CTAs by intent, wireframes annotated.

OUTCOME

A clearer way to say what Trocco does, and a shared language for the team.

[Read the full case study on Framer](#)

Full case studies, with wireframes and visuals, are on my Framer site.

Both projects, plus writing on UX messaging.

uxwithantora.framer.website

The part that does not show up in Figma.

Backlog management

I run my own Jira backlog: 20 to 30 actionable tickets across Highest, High and Medium, sequenced against real engineering dates and grouped by component so parallel work does not collide. I also run structured reviews with engineers, PMs and clinical operations leads across US and India time zones.

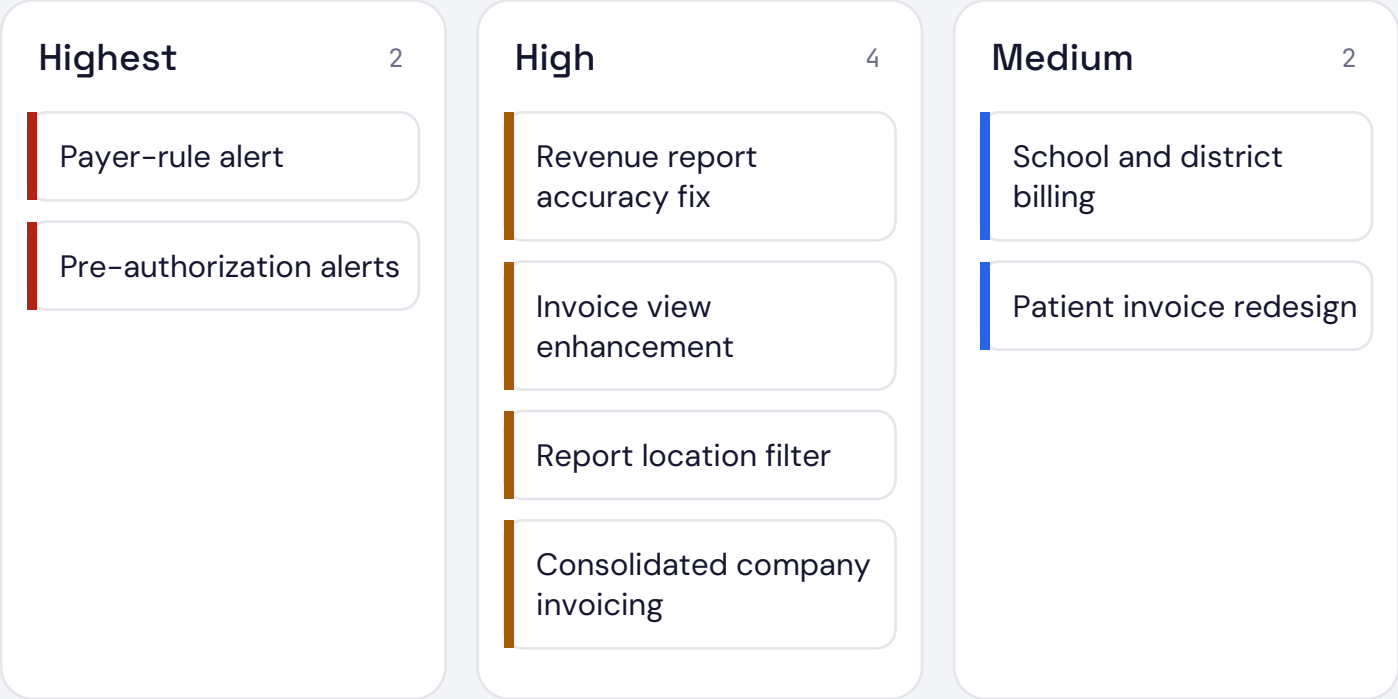
Stakeholder facilitation

On a consolidated invoicing request, I turned one line from a client into a scoped v1: added the contracted-rate override they raised later, deferred payment automation on purpose, and flagged a second ticket that needed the same engine before both got built.

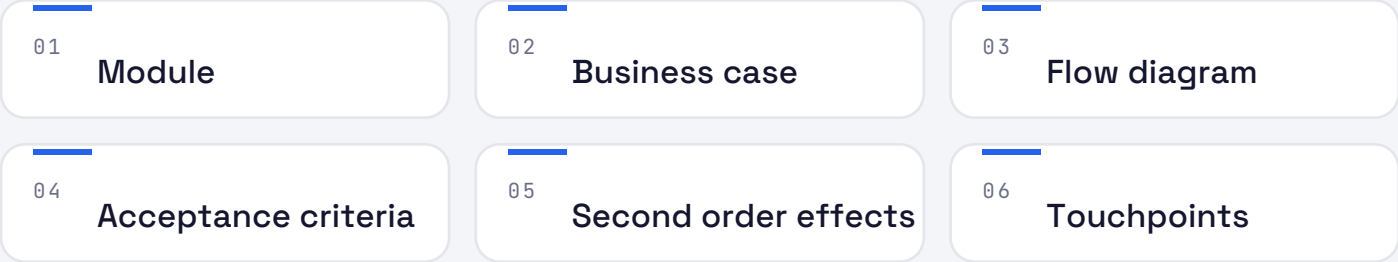
Data over opinion

A team assumed denials came from a stale ICD-10 code list. I traced them with SQL: 97% of non-billable-code denials came from retired codes still selectable in the SOAP note picker. The fix moved from a refresh schedule to the picker.

A SLICE OF THE BACKLOG, THREE TIERS



EVERY PRD, ONE SHAPE



LEADERSHIP

Promoted from UX Designer to Senior UX Designer to UX Manager at Nitrous, shipping 50+ projects at 97% on-time delivery. At WhiteHat Jr I was promoted to Director for quality-managing 500+ live sessions, and mentored and supervised a team of instructors.

What I would carry forward, and what I would change.

Shipping was never the end of the thinking. The honest accounting, case by case.

CASE 01

Patient workspace

LESSON

Missing UI is often missing data modeling. Authorization, POC and claims needed real objects and states before any screen could show them honestly.

TRADEOFF

Patient versus case is handed to the user as a toggle, not resolved. A biller and a therapist would each make that call differently.

WHAT I WOULD DO DIFFERENTLY

Bring the entity model into the same round of role interviews that shaped the IA, not after. State machines go subtly wrong without the people who live in the exceptions.

CASE 02

Unified Billing Dashboard

LESSON

Reference is direction, not a template. My first pass copied reference screenshots literally. Catching that kept each bracket native to its own user.

TRADEOFF

Designing the densest bracket first means the simplest one still carries some of that complexity in its model. I chose maintainability over a perfectly minimal case.

WHAT I WOULD DO DIFFERENTLY

Prototype B7, the eligibility carve-out, earlier. It stress-tested "remove, do not append" hardest, because it drops billing entirely.

CASE 03

SPRY Verify

LESSON

Seven versions is not indecision. Each round was shaped by what operators actually did, until the page was in the right order: numbers, problems, then work.

TRADEOFF

Stage A scans well but its panels read at similar weight. Stage B fixes that, but eligibility-only users now filter past auth tickets they never touch.

WHAT I WOULD DO DIFFERENTLY

Run the 50+ urgent messages and 200+ tickets stress test from version one, not at the end. Hierarchy retrofitted onto four sample messages is much harder.

STILL OPEN IN VERIFY, ON PURPOSE

Priority at equal urgency

Pre-attentive hierarchy

Space utilization

LET'S TALK

Looking for a product designer who can also run the backlog?

I like working with people who are building thoughtful things. If you are exploring an idea, stuck on structure, or want a second brain on your product, say hello.

EMAIL antorac6@gmail.com

LINKEDIN [linkedin.com/in/antorachattopadhyay](https://www.linkedin.com/in/antorachattopadhyay)

FRAMER SITE uxwithantora.framer.website

PORTFOLIO antora-c.github.io/Portfolio

RESUME [View on Google Drive](#)

BOOK [15 minute chat](#)

